

AUTO REPAROS – SISTEMA INTEGRADO DE OFICINA MECÂNICA

Fase 3: Operação Corporativa, Serverless, Banco Gerenciado e Segregação Multi-Repo

FIAP – Pós-Tech Software Architecture (Grupo 78 – Turma 15SOAT)

Repositórios Oficiais & Governança Git

- 1. **AutoReparos.App** [GitHub]
Aplicação Principal: API .NET 10, Domínio DDD, Angular 19.
- 2. **AutoReparos.AuthLambda** [GitHub]
Função Serverless: Validação CPF Módulo 11, consulta DB e JWT efêmero.
- 3. **AutoReparos.Infra.Database** [GitHub]
Terraform IaC: AWS RDS PostgreSQL 16 Multi-AZ, PITR e KMS.
- 4. **AutoReparos.Infra.K8s** [GitHub]
Terraform IaC: Cluster EKS, API Gateway v2, VPC Link e Helm Charts.
- Hub / Orquestrador: **AutoReparos** (Submódulos Git e Compose).

✅ **Confirmação Avaliador:** Usuário soat-architecture adicionado com permissão em todos os 5 repositórios.

Integrantes do Grupo 78

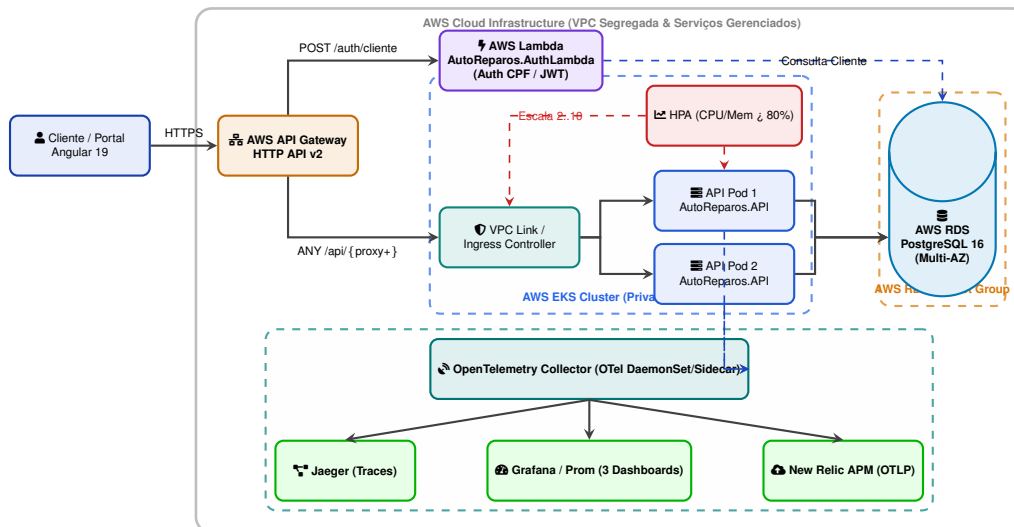
- Enrico Gollner (RM 370737)
- José Dotta (RM 372959)
- Júlia Santos (RM 370364)
- Lucas Bastos (RM 370749)
- Mateus Lecchi (RM 371085)

Vídeo Demonstrativo (Até 15 Minutos)

Link: Vídeo de Apresentação no Google Drive

Demonstração: 4 repositórios, CI/CD, Auth Lambda CPF, APIs protegidas Zero-Trust, 3 dashboards ao vivo no Grafana, Traces e APM New Relic.

Arquitetura Corporativa em Nuvem (AWS API Gateway, Lambda, EKS, RDS e Observabilidade)



Estrutura Cronometrada do Vídeo Demonstrativo (15 Minutos):

Bloco	Janela	Objetivo e Conteúdo Apresentado
1	00:00 - 02:00	Apresentação da equipe Grupo 78 (15SOAT), desafio da oficina e topologia de 4 repositórios Git independentes.
2	02:00 - 05:00	Pipelines de CI/CD automatizadas no GitHub Actions, suíte de testes unitários/integração e governança de PR.
3	05:00 - 07:30	Autenticação Serverless via CPF no API Gateway (POST /auth/cliente), validação Módulo 11 e emissão JWT efêmero.
4	07:30 - 10:00	Consumo de rotas protegidas com JWT e demonstração prática do isolamento Zero-Trust (filtro de placa sem vazamento de dados).
5	10:00 - 13:00	Análise em tempo real dos 3 dashboards mandatórios no Grafana (Volume de OS, Tempo Médio por Status e Erros/Latência).
6	13:00 - 15:00	Rastreamento distribuído de requisições no Jaeger, logs estruturados em JSON e exportação de telemetria ao New Relic APM.

AUTO REPAROS – DEFESA ARQUITETURAL E EVIDÊNCIAS

Documentação Técnica, Requisitos do Edital, Observabilidade e Qualidade

☰ Mapeamento de Conformidade com o Edital (Tech Challenge Fase 3)

Requisito FIAP	Status	Implementação & Evidência Técnica
1. 4 Repositórios Git	Atendido	Repositórios autônomos para App, AuthLambda, Infra.Database e Infra.K8s com CI/CD autônomo.
2. Auth Serverless (CPF)	Atendido	AWS Lambda validando CPF (Módulo 11), checando banco RDS e emitindo JWT de 1h sem sujar Identity.
3. Banco Gerenciado (RDS)	Atendido	Migração de StatefulSet local para AWS RDS PostgreSQL 16 provisionado 100% via Terraform IaC.
4. API Gateway na Borda	Atendido	AWS API Gateway v2 com VPC Link: roteia /auth/cliente para Lambda e /api/* para EKS.
5. Isolamento Zero-Trust	Atendido	Dados do portal filtrados estritamente pela claim ClienteId; busca de placa alheia retorna vazio HTTP 200.
6. CI/CD & Proteção Main	Atendido	Branch main com proteção obrigatória de PR e execução de esteiras automáticas de compilação e testes.
7. 3 Dashboards de Negócio	Atendido	Dashboards Grafana: (1) Volume Diário de OS; (2) Tempo Médio por Status; (3) Falhas em Integrações/DB.
8. APM de Mercado	Atendido	Integração validada com New Relic via OpenTelemetry OTLP (0 falhas em logs, métricas e spans).

📄 Decisões Arquiteturais (RFCs & ADRs)

- **RFC-001 (Multi-Repo & Cloud):** Topologia de VPC AWS, subnets privadas e segregação em 4 repositórios com matriz RACI.
- **RFC-002 (RDS PostgreSQL 16):** Decisão de migração de StatefulSet para banco gerenciado RDS com SLA 99.95%, PITR e KMS.
- **RFC-003 (Auth Serverless):** Arquitetura de autenticação de clientes por CPF/e-mail em Lambda sem contas no Identity.
- **ADR-001 (AWS API Gateway v2):** Escolha de HTTP API v2 com VPC Link para menor latência e custo reduzido vs ALB/Traefik.
- **ADR-002 (Zero-Trust & BOLA):** Mitigação de vulnerabilidades BOLA no portal com validação contextual de claims JWT.
- **ADR-003 (Observabilidade OTel):** Estratégia de telemetria neutra vendor-agnostic exportando para Jaeger e New Relic.
- **Modelo de Dados & ERD:** Justificativa formal do PostgreSQL 16 vs NoSQL/MySQL com dicionário relacional completo.

🔍 Observabilidade & Dashboards Mandatórios

- **Dashboard 1 – Volume Diário de OS:** Monitora total de OS criadas em 30 dias, fluxo diário e evolução de status (Kanban).
- **Dashboard 2 – Tempo Médio por Status:** Acompanha permanência das ordens nas fases de Diagnóstico, Execução e Finalização.
- **Dashboard 3 – Falhas e Integrações:** Mede taxa de erros HTTP 5xx, falhas de notificação e latência de queries no banco RDS.
- **Logs Estruturados em JSON:** Logs contêm atributos correlacionados TraceId e SpanId para rastreabilidade de ponta a ponta.
- **Métricas de Negócio Nativas:** Interceptor de persistência no EF Core garante emissão de métricas em qualquer mutação de status.
- **Validação Real no New Relic:** Carga com zero falhas na autotelemetria do Collector (logs, spans e metric points entregues).

🔧 Suíte de Testes Automatizados (354 Testes – 100% Verde)

Projeto de Testes	Qtd	Resultado
Domain.Tests	113	113 Passaram (0 Falhas)
AuthLambda.Tests	40	40 Passaram (0 Falhas)
Application.Tests	134	134 Passaram (0 Falhas)
IntegrationTests (DB Real)	67	67 Passaram (0 Falhas)
Total Consolidado	354	354 Testes Aprovados

Testes de integração utilizam contêineres reais via Testcontainers, validando concorrência e banco sem mocks.

➤ Como Executar Localmente

```
git clone --recurse-submodules \
https://github.com/Grupo78-PosTech-15SOAT/\
AutoReparos.git
cd AutoReparos && cp .env.example .env
docker compose up -d --build
```

Serviços disponíveis após o boot:

- **Swagger API:** http://localhost:8080/swagger
- **Grafana:** http://localhost:3000 (admin / admin)
- **Jaeger UI:** http://localhost:16686
- **Prometheus:** http://localhost:9090